Real-Time Audiovisual Interaction with Max/MSP Jitter

Fengkai Liu

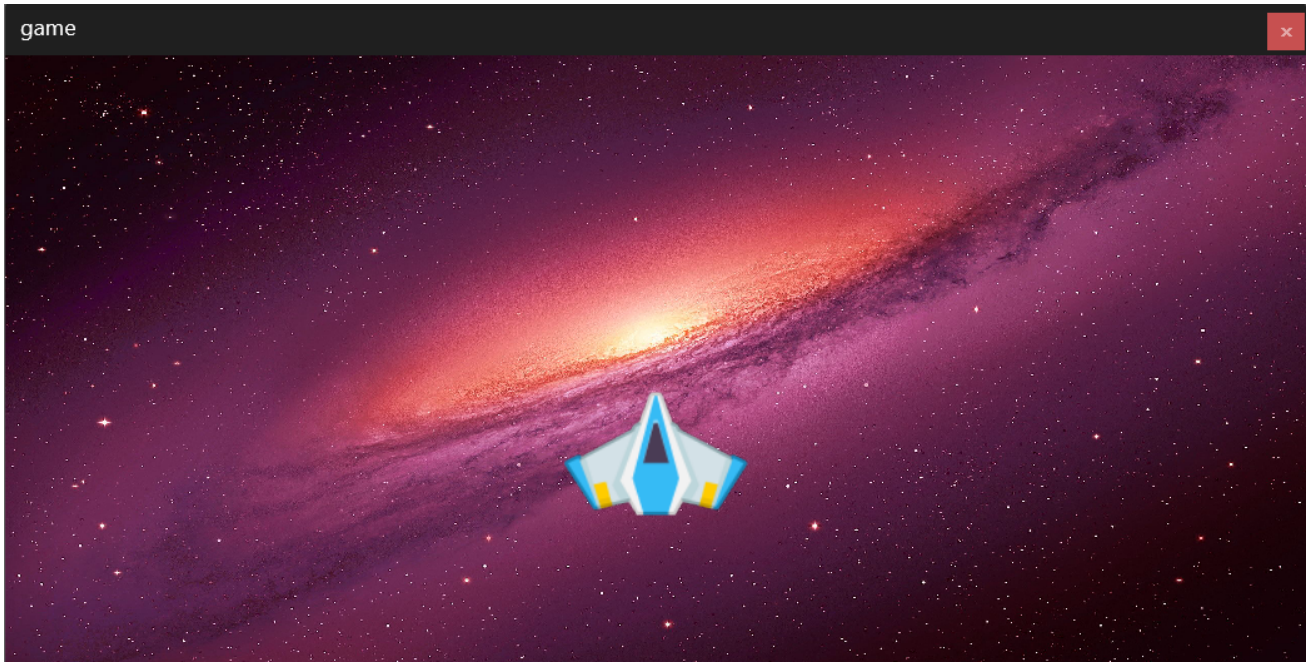


Figure 1: Interface of the game.

Abstract

This article introduces "2D Music Starship", a simple music game that combines graphics and music created by the Max/MSP [1] and Jitter libraries. Players control a 2D interstellar spaceship through keyboard and mouse input. While operating, corresponding notes will be output according to different inputs. This project presents a novel concept of music software that connects the concepts of games, music and. It shows that Max/Jitter can have many possibilities, not limited to music production, which demonstrates the differences of MAX compared to other music software.

Keywords

Jitter, MAX/MSP, JavaScript, 2D Game, Starship, Interactive Audio

1 Introduction

At many music festivals, we can see the combination of graphical interfaces and music. In MAX/MSP, through its Jitter library[1], it can provide creators with a powerful creative platform to build stunning music visual designs. Out of my personal interest in game design and music interaction, this project aims to explore the possibilities of music interaction through a simple game operation interface.

The main goal of "2D Music Starship" is to show users how to complete simple music visual interface interaction through MAX and shape music through different methods.

2 Related Work

Max/MSP with jitter has been widely used in real-time audiovisual performances and interactive applications. Projects like Kevin Kripper's "Vsynth", a modular visual synthesizer [2], demonstrate how Jitter's integration enables artists to apply the concept of sound synthesis to visuals and create complex visual instruments. Furthermore, large-scale interactive games like "Squidball" (SIGGRAPH 2004) [4] highlight Jitter's ability as a library to handle audio and visuals very well.

In addition, tutorial series such as Amitchellaudio's "How to Make Games with Jitter in Max/MSP" [3] demonstrate practical methods for developing game mechanics and visual feedback using JavaScript within the Jitter framework. These creative ideas are constantly expanding the ways in which MAX/MSP can be used. After all, who would have thought of making games through music software?

The project of this article is based on these projects. By studying the implementation methods of different projects, it clearly integrates music interaction directly into the game mechanism, demonstrating the powerful aspect of Max/Jitter as a music tool.

3 Design and Implementation

"2D Music Starship" is entirely developed in Max/MSP, using Jitter for graphic rendering and MSP for audio synthesis. The JavaScript files are used to handle gameplay logic, user input detection, and sprite calls.

This work is licensed under a Creative Commons Attribution 4.0 International License.

NIME '25, June 24–27, 2025, Canberra, Australia

© 2025 Copyright held by the owner/author(s).

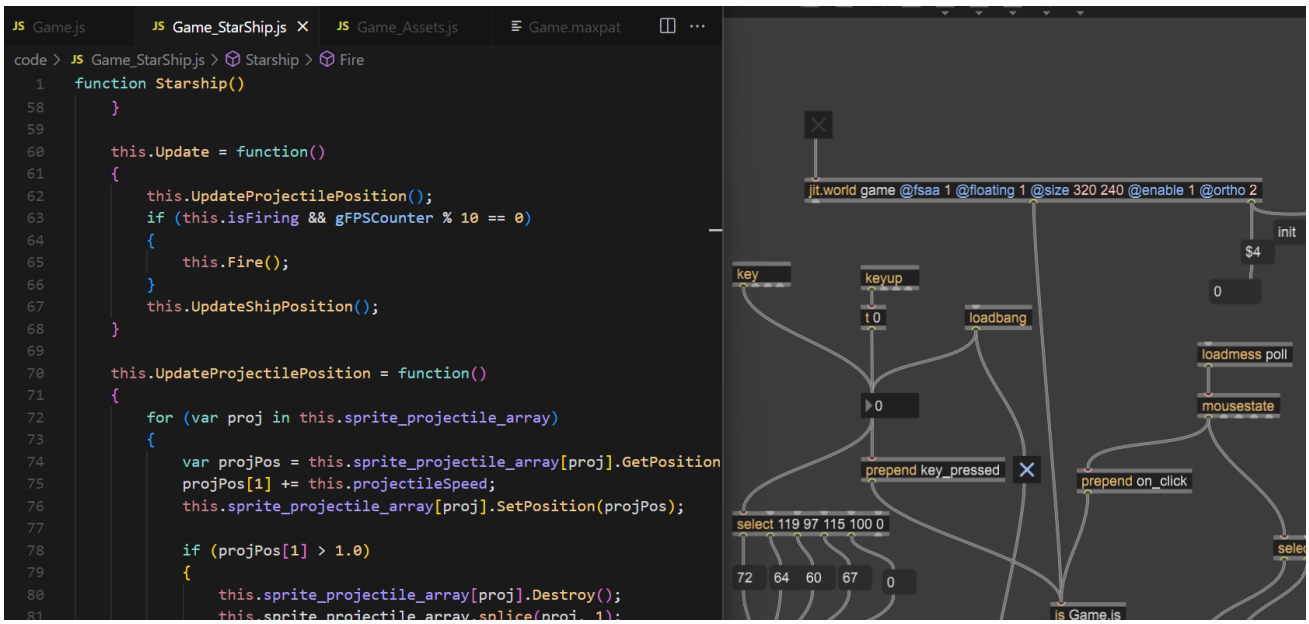


Figure 2: Partial of the inner work.

3.1 Graphics Engine

Implemented using Jitter's `jit.world` object to manage game sprites, background images, and visual effects. Sprites were managed through JavaScript classes for reusability and efficient rendering.

3.2 User Input

Keyboard and mouse inputs were detected in JavaScript, controlling the starship's movement and projectile-flight actions.

```
function key_pressed(key) {
    gPlayerShip.Move(key);
}

function on_click(button) {
    gPlayerShip.SetIsFiring(button);
}
```

3.3 Audio Engine

Keyboard inputs trigger MIDI-to-frequency conversions (`mtof` object in Max), feeding audio signals into `cycle~` objects for continuous wave synthesis. Output was handled through `ezdac~`.

4 Results and Discussion

After testing, the "2D Music Starship" successfully demonstrated the real-time interaction between vision and audio. The player's actions will immediately generate corresponding music, creating dynamic background music related to the gameplay. The players find this audio-visual synchronization very intuitive and attractive, thereby verifying the basic design goals of the project.

This game currently has only one spaceship and the corresponding music output. However, the simplicity of the game provides enough room to increase the depth and complexity of the gameplay. During the development process, a key design challenge was how to integrate the music well into the entire game so that players didn't feel anything strange.

Overall, this project has demonstrated the effectiveness of Max/MSP and Jitter as interactive audiovisual systems, especially suitable for music-focused game design and interactive media. However, through various related materials available online, Jitter can also play a role in many other fields.

5 Conclusions and Future Work

This project successfully showcased the potential to integrate interactive music with simple gameplay mechanics using Max/MSP and Jitter. Key achievements include robust real-time audiovisual synchronization, responsive gameplay mechanics, and the creation of an interactive musical interface.

Future directions involve enhancing the complexity of the game by incorporating advanced mechanics such as AI against enemies, interactive environments, or level progression. Expanding musical complexity, such as procedural melody generation triggered by gameplay, could further deepen the immersion of the player. Furthermore, future iterations could explore multiplayer aspects or public installations, promoting collaborative music making.

Ultimately, '2D Musical Starship' serves as a foundation for future research and creative exploration within interactive musical interfaces, highlighting Max / Jupiter's versatility and potential for broader educational and artistic applications.

References

[1] Cycling '74. *Max/MSP Jitter Documentation*, 2024. Available: https://cycling74.com/docs.
[2] Kevin Kripper. Vsynth: Modular visual synthesizer for max. https://www.kevinkripper.com/vsynth, 2020. Accessed April 2025.
[3] A. Mitchell. How to make a game in max/msp with jitter. https://www.youtube.com/watch?v=14spsUswKMI, 2022. Accessed April 2025.
[4] SIGGRAPH 2004. Squidball project. https://www.siggraph.org/s2004, 2004. Accessed April 2025.